



Degree Programme in Computer Engineering

First Cycle 15 credits

LLM: Retrieval vs. Parametric Memory Tradeoff

A Comparison of Retrieval-Augmented Generation and Standalone Large
Language Models Using RAGAS Answer Accuracy

Asif Rezai & Shirvan Hasan

LLM: Retrieval vs. Parametric Memory Tradeoff

LLM: Hämtning kontra Parametriskt Minne

Asif Rezai
Shirvan Hasan

Degree Project in Computer Engineering
First Cycle, 15 ECTS
Supervisor: Maksims Kornevs
Examiner: Anders Lindström
TRITA-CBH-GRU-2025:095

KTH
School of Technology and Health
141 52 Huddinge, Sweden

Abstract

Large language models (*LLM*) have made remarkable progress but still suffer from factual inconsistencies when it comes to general knowledge questions, often producing incorrect or unsupported information. This thesis addresses that limitation by comparing two paradigms for open-domain question answering: a retrieval-augmented generation (*RAG*) system that supplements a small *LLM* with an external document retriever, and a large standalone *LLM* that relies solely on knowledge stored in its model weights. The central question is whether a much smaller retriever-backed model can achieve factual accuracy comparable to or better than a state-of-the-art large standalone *LLM* in answering general knowledge questions. To investigate this, a *RAG* pipeline was implemented using a relatively small *LLM* (billions of parameters) for generation, and a large (millions of documents) retrieval corpus. Its performance on answer accuracy was then evaluated against leading large *LLMs* (trillion+ parameters) on a curated dataset of 1,000 open-domain question & answer (*QA*) pairs. Each system's rate of factually correct answers were measured and analyzed where their responses differ. This study thus explores the trade-off between “memory” (internal parametric knowledge) and “retrieval” (external knowledge lookup) in *LLMs*, aiming to understand how much external retrieval can compensate for a smaller model's limited internal knowledge in terms of factual accuracy.

Keywords

Retrieval Augmented Generation, RAG, Large Language Models, LLM, Parametric memory, Retrieval, FAISS, ChromaDB

Sammanfattning

Stora språkmodeller (LLM:er) har gjort anmärkningsvärda framsteg men lider fortfarande av faktamässiga inkonsekvenser och producerar ofta felaktig eller obestyrkt information. Denna uppsats tar itu med den begränsningen genom att jämföra två paradig för frågesvar i öppna domäner: ett retrieval-augmented generation (RAG)-system som kompletterar en liten LLM med en extern dokumenthämtare, och en stor parametrisk LLM som enbart förlitar sig på kunskap lagrad i sina modellvikter. Den centrala frågeställningen är huruvida en betydligt mindre modell med RAG-stöd kan uppnå en faktamässig noggrannhet som är jämförbar med, eller bättre än en avancerad stor LLM vid besvarande av allmänkunskapsfrågor. För att undersöka detta implementerades en RAG-pipeline med en relativt liten LLM (med miljarder parametrar) tillsammans med ett textkorpus i webbskala, och dess prestanda utvärderades mot en ledande stor LLM (med biljoner parametrar) på en kuraterad datamängd bestående av cirka 1 000 öppna QA-par. Varje systems andel faktamässigt korrekta svar mättes och analyserades där deras svar skilde sig åt. Denna studie utforskar därmed avvägningen mellan "minne" (intern parametrisk kunskap) och "hämtning" (extern kunskapsuppslagning) i LLM:er, med målet att förstå hur mycket extern hämtning kan kompensera för en mindre modells begränsade interna kunskap när det gäller faktanoggrannhet.

Nyckelord

Hämtningförstärkt generering (RAG), stora språkmodeller (LLM), parametriskt minne, hämtning, FAISS, ChromaDB.

Acknowledgements

This thesis was conducted at KTH Royal Institute of Technology in the field of computer science and engineering.

We extend our deepest gratitude to Maksims Kornevs, our supervisor, for his invaluable guidance, feedback, and unwavering support. His insights have been instrumental to this work.

Finally, we acknowledge all others at KTH and beyond who supported this thesis. Their encouragement and input have been invaluable.

Thank to everyone who contributed to the successful completion of this project.

Table of Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Goal and Objectives	1
1.3	Delimitation	2
1.4	Acknowledgment of AI Assistance	2
2	Background and Theory	3
2.1	Memory and Retrieval in Large Language Models	3
2.1.1	Head-to-Head Comparisons	4
2.1.2	Scaling and System-Level Trade-Offs in Retrieval-Augmented Models	4
2.1.3	Mechanistic Analyses	5
2.1.4	Implications	5
2.2	RAGAS Framework for Automated Evaluation	5
2.3	Empirical Validation of the Answer Accuracy Metric	5
2.3.1	Correlation with Expert Judgements	6
2.3.2	Adoption in Production Tooling	6
2.4	Vector Stores	6
2.4.1	FAISS and Chroma	6
2.5	Impact of top-k and Temperature (T) on AA in RAG QA Systems	7
2.5.1	$top-k$ Retrieval: Balancing Relevance and Noise	7
2.5.2	Temperature T : Diversity vs. Determinism in Generation	7
3	Methodology	9
3.1	Experimental Setup and Tools	9
3.1.1	Model-GPU Matching	9
3.1.2	Inference Framework: Ollama on RunPod	9
3.2	Design a Challenging Open-Domain Factual QA Evaluation Set	10
3.2.1	Starting Corpus – RefinedWeb (CommonCrawl)	10
3.2.2	Factuality Filtering via LLM	10
3.2.3	Synthetic QA Pair Generation	10
3.2.4	Removal of Subjective or Ambiguous Questions	11
3.2.5	Final Review and Dataset Assembly	11
3.2.6	Fairness of Setup	11
3.3	Implementing a RAG pipeline	11
3.4	RAG and Large LLM Evaluation	12
3.4.1	RAG System Evaluation	12
3.4.2	Large LLM Evaluation	12
3.5	Retrieval Quality Comparison	13
3.6	Hyperparameter Sensitivity Analysis	14

4	Results	15
4.1	RAG and Large LLM Evaluation	15
4.2	Retrieval Quality Comparison	16
4.3	Hyperparameter Sensitivity Analysis	17
5	Discussion	19
5.1	Memory vs Retrieval – Results Review	19
5.2	Retrieval Failure analysis	19
5.2.1	Document length Bias	20
5.2.2	Retrieval Methods: Multi-step retrieval	20
5.3	Impact of top-k and temperature on RAG systems	21
5.4	Synthesis, Limitations, and Future Work	21
6	Conclusions	23

1. Introduction

1.1 Problem Statement

Large Language Models (*LLMs*) have shown impressive language generation capabilities but struggle with factual consistency. They often 'hallucinate', confidently generating plausible but incorrect information [1]. A core issue is that traditional *LLMs* rely on parametric memory (knowledge encoded in the model weights), which is static and limited. Once trained, *LLM*'s knowledge is essentially 'frozen in time', leading to stale knowledge of recent events and difficulty with specialized or long-tail facts [2]. Without an internal fact-checking mechanism, *LLM* can interpolate noisy training data or fill gaps by inventing details, resulting in statements that lack verification against reality [1]. Recent efforts to mitigate hallucinations (e.g., fine-tuning or prompting techniques) have made progress, but gaps remain, especially for open-domain questions where any topic might appear. This thesis explores retrieval-based methods as a potential approach to address these gaps and presents preliminary conclusions based on the investigation. One such method involves integrating an external knowledge source to ground the *LLM*'s answers in up-to-date evidence. In this context, Retrieval-Augmented Generation (*RAG*) systems have emerged as a promising approach to reduce hallucinations. [3]. Although it is not fully understood how a smaller *RAG*-backed model compares to a massively large standalone *LLM* (like *GPT-4*) on factual questions & answers (*QA*). The central research question is thus: Can a retrieval-augmented *LLM* with far fewer parameters outperform a much larger parametric-only *LLM* in factual accuracy for open-domain *QA*? This problem statement highlights the tension between storing knowledge in the model (memory) versus fetching it as needed (retrieval), and motivates a thorough investigation into this memory vs. retrieval tradeoff.

1.2 Goal and Objectives

The primary goal of this study is to compare factual accuracy, retrieval quality and hyperparameter sensitivity of a *RAG* system built on a relatively smaller *LLM*, against a state-of-the-art large *LLM* on open-domain question answering. This goal can be broken down into five concrete objectives:

- **Design a Challenging Open-Domain Factual QA Evaluation Set:** Build a diverse dataset of open-domain *QA* pairs, each with human-verified factual answers, to serve as a rigorous benchmark for evaluating *QA* systems..
- **Implement a RAG Pipeline:** Build a question answering system where a small-sized *LLM* is augmented with a retrieval module. This involves integrating a document retriever (over a large text corpus) with an *LLM* generator that uses the retrieved evidence to answer questions. To ensure a fair comparison with the *LLM*'s parametric memory, the system is tuned to rely exclusively on the retrieved context as its factual source, maximizing both factual correctness and source attribution.

- **RAG and Large LLM Evaluation:** Evaluate, in factual accuracy, the *RAG* system’s performance versus the performance of a much larger standalone model that relies only on parametric knowledge.
- **Retrieval Quality Comparison:** Quantify and compare the retrieval recall of two vector search backends FAISS and ChromaDB, on a controlled 1,000-QA dataset, thereby establishing whether retrieval quality constitutes a primary bottleneck in the studied RAG pipeline’s answer accuracy and informing where optimization efforts should be directed (retriever vs. embeddings/other factors).
- **Hyperparameter Sensitivity Analysis:** Quantify how top-k and generation temperature (T) affect the robustness and answer accuracy of the *RAG* pipeline, and to identify hyperparameter settings that deliver the best performance on 1000 QA test set.

1.3 Delimitation

To maintain feasibility and focus, this thesis defines a clear scope and set of limitations. The study is restricted to open-domain QA in English, targeting general knowledge across a broad range of topics, excluding domain-specific or multilingual settings.

The retrieval corpus is based on CommonCrawl web data (RefinedWeb) [4], consisting solely of plain text. Specialized databases and multi-modal sources are excluded. This design ensures that both the parametric *LLM* and the *RAG* system access knowledge from the same distribution, supporting fairness in comparison.

The evaluation focuses exclusively on factual accuracy, omitting other dimensions such as fluency, coherence, or style. The evaluation set comprises approximately 1,000 curated QA pairs, selected to enable manual fact-checking. The knowledge scope is limited to content available up to the CommonCrawl snapshot date to avoid discrepancies in model training cutoffs.

These constraints, support a controlled and fair comparison between parametric memory and retrieval-based methods.

1.4 Acknowledgment of AI Assistance

During the writing of this thesis, AI tools, including ChatGPT by OpenAI, were used to assist with grammar correction, language refinement, and writing clarity. All substantial content, analysis, and conclusions are the result of the author’s own research and critical thinking. The use of AI was limited to editorial support and did not influence the intellectual or conceptual development of the work.

2. Background and Theory

Recent advances in large language models (*LLMs*) have enabled impressive capabilities across a wide range of natural language understanding and generation tasks. However, these models still face fundamental limitations in terms of factual accuracy, knowledge coverage, and up-to-date information access. This chapter surveys the theoretical and empirical foundations that motivate the use of Retrieval-Augmented Generation (*RAG*) as a hybrid solution to these challenges.

It begins by outlining the trade-offs between parametric memory and external retrieval in transformer-based models, highlighting the shortcomings of purely parametric approaches in terms of scalability, staleness, and hallucination risk. It then presents comparative studies showing how retrieval-augmented systems can outperform much larger (*LLMs*) on knowledge-intensive tasks. This is followed by an overview of the *RAGAS* framework, which enables automated evaluation of *RAG* pipelines, and a discussion of the Answer Accuracy metric, which correlates strongly with human judgments. Finally, it examines the role of vector stores and how system-level parameters like top-k and temperature affect answer quality in practice. Together, these topics establish the theoretical and technical grounding for the experimental work presented later in this thesis.

2.1 Memory and Retrieval in Large Language Models

Transformer-based language models, such as *GPT-4*, acquire an implicit "parametric memory" during pre-training, storing vast amounts of world knowledge directly in their weights [5]. This built-in store allows them to answer many questions instantly and without external resources. Yet its capacity is uneven: frequent facts encountered often in the training corpus are recalled reliably, whereas rarer and niche (long-tail), facts frequently fall below the memorization threshold, even when model size is increased dramatically [6]. Pushing capacity further is possible only by training ever larger networks on ever larger datasets, a strategy that quickly encounters steep computational cost and diminishing returns.

The static nature of parametric memory introduces two further challenges. First, knowledge "freezes" at the moment training stops, as real-world facts evolve, answers grow stale unless the model is regularly retrained or carefully fine-tuned [2]. Second, next-token prediction rewards plausibility rather than certainty, so gaps in coverage can be filled by fluent but fabricated statements, so called hallucinations, without any built-in mechanism for verification [1]. Together, limited coverage, staleness, and unverifiable generations constrain the reliability of pure parametric approaches.

External retrieval offers a complementary path. Instead of relying solely on what is baked into the weights, the model can query a much larger, continually updated corpus at inference time and condition its generation on the retrieved passages. *RAG* has been shown to let a modest-sized model match or exceed the factual accuracy of far larger stand-alone *LLMs* on knowledge-intensive tasks [5, 6]. By coupling on-demand lookup with smaller neural networks, practitioners trade storage inside the model for storage in an external index, gaining broader

coverage and fresher information at a fraction of the training cost.

This shift is not without drawbacks. Retrieval adds latency, introduces new failure modes (for instance, retrieving irrelevant or incorrect documents), and complicates the overall system design. Nevertheless, a growing body of evidence indicates that beyond a certain model size, allocating additional compute to retrieval yields better factual accuracy per unit of compute than further enlarging parametric memory. More on this in (Section 2.1.2).

2.1.1 Head-to-Head Comparisons

Smaller models enhanced with retrieval capabilities have demonstrated competitive or even superior performance compared to significantly larger standalone language models. As presented in (Table 2.1), Atlas 11B model outperforms the much larger PaLM-540B on the Natural Questions benchmark (42% vs. 39%) while having approximately fifty times fewer parameters[7]. Similarly, RETRO 7.5B achieves higher accuracy than Jurassic-1-175B and Gopher-280B on the majority of Pile tasks, despite the substantial difference in model size [8]. Additionally, a GPT-Neo 2.7B model augmented with dense retrieval answers long-tail PopQA questions more than twice as accurately as GPT-3-davinci-003-175B (41% vs. 19%) at a fraction of the inference cost [6].

Table 2.1: Comparison between retrieval-augmented models and larger parametric models [7, 8, 6].

Retrieval-augmented model	Parametric model it beats	Key metric	Compute gap
Atlas 11 B	PaLM 540 B	Natural Questions, 42% vs 39% accuracy	$\approx 50\times$ fewer parameters
RETRO 7.5 B	Jurassic-1 175 B & Gopher 280 B	Beats the larger models on 10/16 and 9/16 Pile tasks	$\approx 25\times$ fewer parameters
GPT-Neo 2.7 B + dense retrieval	GPT-3 davinci-003 175 B	Long-tail PopQA: 41% vs 19% accuracy	$\approx 65\times$ fewer parameters and far lower inference cost

2.1.2 Scaling and System-Level Trade-Offs in Retrieval-Augmented Models

A recent study Shao et al. (2024) highlight both theoretical and practical advantages of retrieval augmentation across different axes of model development [9]. They derive compute-optimal curves, which show how to best allocate computational resources for training. Their findings suggest that once total training cost, measured in floating-point operations (*FLOPs*) a standard unit for estimating compute, exceed $\sim 10^{23}$ then diverting additional compute from increasing Transformer depth & adding parameters, to data-store size & retrieval steps improves accuracy more steeply. Independent scaling studies of *RETRO* and *Atlas* report similar crossover points [7, 8].

At the system level, latency-aware evaluations also favor retrieval-based architectures. Shen et al. (2024) compares a 9B Gemma model with and without a *RAG* pipeline to a 34B parametric baseline, showing that even with a 40% latency increase, the *RAG*-enhanced system achieves 30% higher “accuracy-per-GPU-hour” on TriviaQA[10].

2.1.3 Mechanistic Analyses

Mechanistic probes reveal how retrieval compensates for parametric gaps. Farahani et al. (2024) show that Atlas dynamically prefers knowledge in retrieved passages whenever they conflict with parametric-knowledge, precisely in cases where memorization is weakest[11]. Surveys synthesizing over one hundred *RAG* papers concur that retrieval becomes the dominant path to further factual accuracy gains once surprising or qualitatively new skills that weren't present in smaller models appear in 5–10B-parameter models [12]. Further empirical results bolster this view. Krishna et al. (2024) report that augmenting *LLMs* with a multi-step retrieval pipeline boosts accuracy on complex questions by 50% compared to parametric-only baselines[13]. Similarly, Tran et al. (2024) demonstrate that fact-checked retrieved contexts significantly enhance reliability in medical question answering tasks[14]. Together, these findings underscore the role of retrieval in mitigating hallucinations and enhancing factual accuracy in knowledge-intensive domains.

2.1.4 Implications

Collectively, these studies support the thesis that *RAG* systems can realize higher factual accuracy at lower compute budgets by combining moderate-sized language models with scalable, updatable external memory. The remainder of this thesis builds on that premise, arguing that such hybrid architectures mitigate the limitations of parametric-knowledge while controlling cost and maintaining interpretability.

2.2 RAGAS Framework for Automated Evaluation

RAGAS (Retrieval-Augmented Generation Assessment) is an open-source framework, Python library and research effort that provides an end-to-end, largely reference-free way to quantitatively assess *RAG* systems. It combines *LLM* “critics” with task-specific heuristics to score both the retriever and the generator, giving developers fast, objective feedback without waiting for human labels[15]. By packaging these metrics in a lightweight API and integrating with popular orchestration stacks such as LangChain and LlamaIndex, RAGAS has become the de-facto baseline for *RAG* evaluation workflows in industry and research[16, 17, 18]. LangChain is a framework for building applications with language models by chaining components like prompts, memory, and tools, while LlamaIndex is a data framework that connects *LLMs* to external data sources for indexing and querying documents—both playing key roles in enabling Retrieval-Augmented Generation (*RAG*) systems.

The framework was introduced in late 2023 by Shahul Es, Jithin James et al. in a paper which argued that existing information retrieval and question answering evaluation metrics correlate poorly with human judgments for *RAG* pipelines[15]. Because a *RAG* system has two loosely coupled components, retrieval and generation, traditional single-score metrics obscure where quality issues really arise, RAGAS therefore set out to measure each stage separately and to do so with minimal or no labeled data[19].

2.3 Empirical Validation of the Answer Accuracy Metric

Accurately assessing the quality of generated answers in *RAG* systems is critical for evaluating their usefulness and reliability. The Answer Accuracy (*AA*) metric aims to provide an automated, reference-based evaluation, and uses *LLM* as judge that closely mimics human judgment. By comparing generated answers to ground-truth references and scoring both factual correctness and completeness, each providing a score of 0, 2, or 4. These scores are then normalized to a $[0, 1]$ scale and averaged to produce the final metric. A higher score reflects a closer match between the model's answer and the reference. This metric offers a scalable solution for answer validation in *RAG* pipelines [20].

2.3.1 Correlation with Expert Judgements

A growing body of evidence shows that *AA* achieves a high agreement with subject-matter experts among open-source *RAG* evaluation metrics.

- **Zero-shot leaderboard.** The official RAGAS leaderboard reports a correlation of ≈ 0.90 between *AA* (judged by *mixtral-8 $\times$ 22B*) and human judges [21].
- **Domain evaluation.** In a study by Sujoy et al. (2024), *AA* (labeled “Answer Correctness”) emerged along with *Factual Correctness* as “best aligned with subject matter expert verdicts” on 700+ specialised questions[22].

2.3.2 Adoption in Production Tooling

AA is now the *default headline* score in several industrial evaluation stacks:

1. **LangSmith & LangChain:** the “Evaluate a *RAG* application” tutorial recommends *AA* for response grading.[23]
2. **Gemini 1.5 vs 2.0 flash Benchmark:** *AA* discriminated two model versions on QASPER, a QA dataset on scientific research papers. Gemini 1.5 flash scored an accuracy of 0.542 while Gemini 2.0 flash scored 0.567. [24]
3. **Open-source guides:** Qdrant’s evaluation white-paper and Evidently AI’s “LLM evaluation metrics” guide both single out *AA*/Answer Correctness as the quickest sanity-check for- *RAG* pipelines.[25, 26]

Empirical studies and widespread industry uptake indicate that *AA* offers reliable single-number proxy for human factuality assessment when reference answers are available, making it a sound choice for comparing *GPT-4* against smaller (13–27 B) models paired with retrieval module in the experiments.

2.4 Vector Stores

A critical component in the design of *RAG* systems is the vector store, which is used to store and retrieve relevant documents or chunks. The documents are stored in the form of vectors/embeddings, which capture the semantic meaning of text and enable efficient similarity searches. This is essential when searching for relevant documents to a query to support the generation of factually accurate answers[27, 28].

2.4.1 FAISS and Chroma

Two widely used tools in this domain are Facebook AI Similarity Search (*FAISS*) and Chroma, both of which serve as efficient and scalable vector databases tailored for semantic search. *FAISS*, developed by Meta AI, is an open-source library optimized for high-speed similarity search and clustering of dense vectors. It supports various similarity measures such as Euclidean distance and cosine similarity, and is designed to scale to datasets that exceed the capacity of main memory. In the context of *RAG* systems, *FAISS* plays a key role in enabling fast and accurate retrieval of relevant documents, thus grounding the generated responses in pertinent information [29].

Chroma, on the other hand, is an open-source vector database designed specifically for AI applications. It offers an integrated interface for storing, retrieving, and managing document embeddings. Chroma supports advanced features such as semantic filtering and contextual memory, making it especially well-suited for building *RAG*-based systems. These capabilities enhance the relevance and factual accuracy of generated responses by ensuring that the retrieval

step delivers high-quality context to the language model [30, 31].

Both tools are integral to the retrieval component in *RAG* architectures, enabling efficient and scalable search of semantically relevant documents that underpin the quality of the final generated answers.

2.5 Impact of top-k and Temperature (T) on *AA* in *RAG* QA Systems

RAG systems for open-domain QA use an initial retrieval step (fetching top- k documents relevant to a query) followed by a generation step (using an LLM to produce an answer from the retrieved content). The parameters top- k (number of documents retrieved) and temperature T (randomness in decoding) critically influence answer accuracy. Below, each is discussed in turn, followed by an examination of their trade-offs and a presentation of empirical evidence from benchmarks (Natural Questions, TriviaQA, WebGPT) demonstrating these effects.

2.5.1 top- k Retrieval: Balancing Relevance and Noise

What is top- k ? In *RAG*, the retriever returns the k most relevant documents (passages) for the question. top- k thus controls the amount of context given to the generator. A higher top- k increases the chance that the correct answer-bearing document is included, but also brings in more irrelevant or less useful passages.

As top- k increases, recall improves, the likelihood that at least one retrieved document contains the answer goes up. Recall, in this context, refers to the proportion of questions for which the system successfully retrieves at least one relevant passage that contains the correct answer. For example, on the Natural Questions benchmark, retrieving 20 passages instead of 5 raises recall from about 66–67% to $\approx 78\%$, and using 100 passages reaches $\approx 85\%$ recall [32]. In other words, a low top- k might miss the needed evidence entirely, capping the system’s potential accuracy. Retrieving more documents provides broader coverage of possible answers.

However, a larger top- k also means more noise. Additional documents that may be topically related but not actually useful (or even misleading). The generative reader must sift through more information, which can be detrimental if some passages conflict or distract. Empirical studies have found that beyond a certain point, adding more passages can hurt accuracy. Even relevant but non-evidential documents can change a previously correct answer to an incorrect one, a phenomenon dubbed “*damaging retrieval*”. In other words, too much context can confuse the model. A recent analysis showed that providing all retrieved passages (without filtering) significantly degraded QA performance compared to using only a carefully chosen subset. In fact, filtering out “detrimental” passages improved exact-match accuracy by up to 10% on Natural Questions and TriviaQA. This challenges the assumption that simply giving the model more data is always better [33].

2.5.2 Temperature T : Diversity vs. Determinism in Generation

After retrieval, the generator (an LLM) must formulate an answer from the evidence. The T parameter in decoding controls the randomness of this process. In technical terms, T rescales the model’s output probabilities: low T (approaching 0) makes the model highly deterministic (it will choose the highest-probability next token each time), whereas higher T (e.g. 0.6 or above) makes it more likely to sample less probable tokens, increasing diversity in the output.

For factual QA, precise, correct answers are preferred over creative or unexpected phrasing. A lower temperature typically yields more accurate and on-topic answers. The model will stick to the most likely completion, which, given the presence of relevant context, often means it will

more directly use the evidence provided. As a result, low- T outputs tend to be focused, concise, and factual [34, 35]. This is why many QA systems simply use greedy decoding ($T = 0$ or near zero): it reduces the chance of the model introducing extraneous or incorrect information. In practical guidance, it’s recommended to use a low T for tasks like fact-based QA to “encourage more factual and concise responses” [35].

Temperature tuning in QA is essentially balancing determinism (repeatability and precision) against diversity (richness of expression). Very low T (or using deterministic decoding/beam search) maximizes the chance the model gives a consistent evidence-grounded answer, which is crucial for answer accuracy. The downside is that the answer might be terse or formulaic (since the model always picks the top tokens, it might produce a generic-sounding answer or omit nuance). Slightly higher T can sometimes yield a more explanatory answer or rephrase evidence in an understandable way, which might improve readability or completeness if the model is still grounded in the context. But push T too high and the model may start introducing unsupported claims. In general, for open-domain QA lower temperatures are favored to ensure factuality, whereas high temperatures are avoided because “hallucination ... is a balance between creativity and accuracy” [36]. Many RAG pipelines simply set $T = 0$ (greedy) or a low value like 0.1–0.3 to keep the generation on track [35]. This makes the system more deterministic, if the evidence is there, the model will likely use it to produce the answer, and if not, a low- T model is less likely to fabricate a detailed guess (it might instead give a vague or no answer, which for accuracy is preferable to a confident-sounding wrong answer).

3. Methodology

This chapter outlines the experimental design, tools, and evaluation procedures employed in this study. The primary objective was to compare the factual accuracy of large language models (*LLMs*) and retrieval-augmented generation (*RAG*) systems under controlled and reproducible conditions. To that end, a multi-stage methodology was developed: first, a challenging open-domain factual QA dataset was constructed from scratch. Second, a standardized inference environment was established using containerized GPU instances on the RunPod platform, and third, both (*RAG*) and standalone (*LLMs*) were evaluated using the same question set and objective metrics.

By combining a purpose-built evaluation dataset, consistent inference setups, and rigorous performance metrics, this methodology aims to provide a fair and interpretable comparison of memory-based and retrieval-based approaches to factual question answering.

3.1 Experimental Setup and Tools

All compatibility and performance tests were conducted on the RunPod cloud platform, which provides on-demand access to diverse high-end GPUs and containerized environments. RunPod is a cloud computing service designed for AI and machine learning workloads, offering “enterprise-grade GPUs” [37]. The wide GPU selection and competitive pricing ensured that the experiments were both cost-effective and reproducible. In each RunPod instance the necessary software stack and models were installed within an isolated container image to guarantee consistency across trials.

3.1.1 Model–GPU Matching

GPUs whose VRAM capacity met or exceeded the requirement were assigned for the model being inferenced. In other words, a given GPU was chosen only if its memory (e.g. 24 GB, 48 GB, 192 GB) was greater than the model’s size plus a safety margin. In practice, models up to roughly 12–27B parameters were placed on 24–48 GB GPUs (A5000 and A6000) [38, 39] with room to spare, while larger 70 B models on A100 SXM [40], and even larger models on MI300X [41].

3.1.2 Inference Framework: Ollama on RunPod

For all inference tests and model executions, we used the Ollama framework on the provisioned GPU instances. Ollama is a lightweight local deployment tool that allows running large language models on-premises via a simple command-line interface [42]. Each RunPod VM was configured with an Ollama environment that managed the model files and runtime. Models were installed and run using Ollama, which automatically download and prepare the specified model. Ollama streamlines deployment by containerizing the model environment, so no custom Docker or server setup was needed.

3.2 Design a Challenging Open-Domain Factual QA Evaluation Set

The objective was to develop a challenging, fact-based QA dataset to evaluate factual consistency in language models. A custom dataset was constructed, as opposed to relying on existing QA benchmarks, to ensure alignment with the knowledge source and to facilitate a controlled comparison between parametric models and *RAG* systems. The dataset construction pipeline was inspired by the methodology presented in Pinecone’s *RAG makes LLMs better and equal* [43], which outlines a systematic approach for building open-domain QA datasets grounded in a retrieval corpus. While the core steps—corpus selection, QA generation, and filtering—follow the structure of that work, the current study introduces several key modifications. Notably, different *LLMs* were employed for both QA generation and factuality filtering, with the `llama3.1:8b-instruct-fp16` and `llama3.1:70b-instruct` models replacing those used in the original benchmark `gpt-3.5-turbo` and `gpt-4-turbo`. The *Llama 3.1* is a new open-source *LLM* from Meta available in 8B, 70B and 405B parameter sizes [44]. As for the reasoning behind the model swap, it had to be done in order to stay within budget for this study.

3.2.1 Starting Corpus – RefinedWeb (CommonCrawl)

The initial corpus comprised 8M samples derived from the 980M -document RefinedWeb dataset (*tiuuue/falcon-refinedweb*) [45], an open-domain collection derived from CommonCrawl web data [4]. RefinedWeb is a filtered and de-duplicated version of CommonCrawl that has been used in systems such as Falcon LLM [46].

Each document was split into non-overlapping 512-token chunks, producing 12M total chunks. Because of resource constraints 2M chunks were randomly selected from this pool. This 2M -document subset served both as, the candidate knowledge source for automatic question-answer (QA) generation, and the retrieval corpus for the RAG system.

For QA generation, a further 50 000 random sample documents were picked from the 2M -document subset. Using CommonCrawl data ensures broad topical coverage while maintaining compatibility with the training data of large models such as GPT-4, thereby supporting a fair experimental setup.

3.2.2 Factuality Filtering via LLM

To ensure that the 50 000 retained documents were rich in verifiable factual content, a factuality filtering step was applied. The `llama3.1:8b-instruct-fp16` model was employed to assign scores (1-5) to each document. This model estimates how verifiable and objective the statements within a passage are, potentially identifying and down-ranking opinion-based or speculative content. Documents ranked below 4 were excluded, retaining approximately 15,000 documents with the highest factuality. This step eliminated overly generic documents, while preserving informative sources.

3.2.3 Synthetic QA Pair Generation

An even larger language model `llama3.1:70b-instruct` was then employed to generate QA pairs from the curated corpus. For each document, the model was prompted to produce a QA pair based on the document content. Prompts encouraged a variety of question types (e.g., who, what, when, where, how) and required that answers be either explicitly present in the text or directly inferable. The use of a large LLM ensured diversity and realism in question phrasing, resembling natural human inquiry rather than templated formats.

3.2.4 Removal of Subjective or Ambiguous Questions

Post-generation, the QA pairs underwent a filtering process to eliminate questions that were subjective, speculative, or ambiguous. Examples include queries such as “Why is X important?” or “What might happen if...,” which lack a singular, verifiable answer. A combination of rule-based heuristics and an LLM-based classifier (the same `llama3.1:70b-instruct` model) was employed to detect such questions. Additionally, the classifier was prompted to evaluate whether a question requested factual information or was opinion-based. Only questions with definitive, document-supported answers were retained. Questions for which the generating model could not identify a specific answer were also discarded. After this refinement, a final set of approximately 8,000 high-quality QA pairs remained, from which a 1000 was randomly picked.

3.2.5 Final Review and Dataset Assembly

The final QA pairs were subjected to a manual human review to ensure the accuracy of the answers relative to their source documents. As a result of this review, 140 documents were removed and replaced with correct ones. Each QA pair comprises a question, its correct answer, and a reference to the supporting document. The assembled QA dataset forms the evaluation benchmark for both the RAG systems and standalone, high parameter count, LLMs.

3.2.6 Fairness of Setup

To ensure a fair evaluation, both the LLMs and the RAG systems were tested using the same knowledge domain. The LLM leverages its internal parametric knowledge (trained on large-scale web data, including CommonCrawl), while the RAG system retrieves answers from the externally curated CommonCrawl-derived corpus. By restricting the evaluation to QA pairs grounded in this corpus, potential biases from proprietary or unknown data sources are minimized. Consequently, any performance differences can be more confidently attributed to the retrieval versus memorization capabilities of the respective systems. If the LLM fails to answer a question, it is likely due to parametric limitations rather than lack of exposure, while RAG failures can be indicative of retrieval, ranking issues or possibly the models inability to staying true to the provided context. This design supports a controlled and interpretable comparison between memory-based and retrieval-based approaches. With the dataset finalized and the RAG evaluation framework in place, the groundwork for the experimental phase of this research is complete.

3.3 Implementing a RAG pipeline

After constructing the evaluation QA dataset and document corpus (Section 3.1), a RAG pipeline was implemented to answer the 1,000 questions using the 2-million document collection. The first step was to build a vector index of the corpus for efficient retrieval. All documents were embedded into dense vectors using the open-source Nomic text embedding model (`nomic-embed-text`), which supports long input context and high-dimensional embeddings [47]. These vectors were then indexed with FAISS. This provided a scalable retrieval backend for the RAG system while maintaining reasonable recall of relevant documents. All vector similarity searches were performed with inner-product cosine similarity on normalized embeddings, and the index was built to treat each document chunk as a retrievable unit.

For the generative component of the RAG pipeline, four different LLMs were integrated and evaluated under identical retrieval settings. Each model was used in turn to generate answers given the retrieved context, allowing a comparative analysis of their performance in the RAG setup. The models evaluated in the RAG scenario were:

- **Qwen 2.5 (14B)**: a 14-billion-parameter instruct-tuned model.

- **GPT-3.5-Turbo (Jan 2025)**: OpenAI’s GPT-3.5 Turbo model (2025-01-25 version) via API.
- **Gemma 3 (27B)**: a 27-billion-parameter instruction-tuned Gemma model.
- **Gemma 3 (12B)**: a 12-billion-parameter instruction-tuned Gemma model.

3.4 RAG and Large LLM Evaluation

3.4.1 RAG System Evaluation

Each question from the dataset was posed to the RAG system in the same manner. The query was first encoded into the vector space and the top 3 most similar documents were retrieved from the FAISS index (i.e. $top_k = 3$). This value was selected as a reasonable default, providing a balance between retrieval relevance and computational efficiency, rather than being the result of exhaustive tuning. These three retrieved text passages were then prepended to the question as additional context for the LLM. The generative model was prompted to formulate an answer using only the provided context, ensuring that any factual claims could be grounded in the retrieved documents.

For generation all parameters (e.g. decoding settings) were kept to default consistently across the four model variants to ensure a fair comparison. This uniform setup means that any performance differences can be attributed to the models themselves rather than to changes in retrieval or prompting strategy.

All answers produced by the RAG pipeline were evaluated for correctness against the known ground-truth answers in the dataset. The RAGAS evaluation framework was employed, specifically the *Answer Accuracy* metric, to quantitatively assess each response. As previously mentioned in (Section 2.3.1), this metric shows a strong correlation with expert human judgments and was therefore selected as the metric of choice. It uses an *LLM-as-a-judge* approach to compare the model’s answer with the reference answer, yielding a score between 0 and 1 (with 1 denoting a perfectly correct answer). For every question, the reference (“ground-truth”) answer was taken from the original source document, the same document used to create the QA pair, therefore the metric remained applicable even when no retrieved context was provided.

To report model-level performance, the individual scores were averaged across all $N = 1000$ open-domain QA pairs:

$$\overline{\text{AAs}} = \frac{1}{N} \sum_{i=1}^N \text{AA}(\hat{a}_i, a_i^*), \quad N = 1000.$$

Here, $\hat{a}_i$ is the model’s answer to the i -th question, a_i^* is the corresponding ground-truth, and $\text{AA}() \in [0, 1] \subset \mathbb{R}$ is the RAGAS AA score for that pair. The resulting AA average score ($\overline{\text{AAs}}$) was recorded for each of the four RAG configurations, ensuring a consistent, reference-based metric and a fair evaluation of the system’s performance.

3.4.2 Large LLM Evaluation

In addition to the RAG-based experiments, a parallel evaluation was conducted using several state-of-the-art large language models without retrieval augmentation. This serves as a baseline to compare how well purely parametric knowledge models can answer the same set of 1,000 questions. Three powerful LLMs were selected for this direct QA evaluation:

- **GPT-4.1 (2025-04-14)**: the April 2025 version of OpenAI’s GPT-4, representing a cutting-edge closed-source model.

- **GPT-4 (2024-11-20)**: an earlier GPT-4 variant (November 2024), included to observe performance of a slightly older iteration.
- **Gemini 2.5 (Pro Preview 05-06)**: Google’s Gemini model (v2.5), a next-generation multimodal LLM in preview release (as of 2025-05-06).

Each of these models was presented with the same 1,000 questions used in the RAG evaluation, but without any retrieved documents or external context. The models answered the questions based solely on their internal knowledge and reasoning, effectively performing open-domain question answering. To ensure fairness in the comparison, the prompting format and constraints were kept analogous to the RAG scenario. In practice, this meant each model received the question in a neutral prompt (e.g., “Q: [question]? A:” or an instruct-style prompt) with no additional information, and was asked to provide a concise factual answer. No system messages or few-shot examples were given, so as not to advantage the standalone LLMs with extra context beyond what the RAG models received (aside from retrieved documents).

After obtaining the answers from the three baseline LLMs, the same evaluation procedure was applied as for the RAG system. Each answer was compared to the ground-truth reference answer from the dataset, and the RAGAS Answer Accuracy metric was computed. This means that even though the baseline LLMs did not retrieve documents, their answers were judged against the ideal answer from the corresponding source document (just as with the RAG models). By using identical questions and the same automated accuracy metric, a direct and fair performance comparison was ensured between retrieval-augmented models and large parametric-only models. The outcome of this evaluation (reported in Chapter 4) would reveal how close the RAG approach with smaller models can come to the performance of cutting-edge LLMs, under a controlled and equitable setup. All models were evaluated under the same conditions, and any differences in Answer Accuracy can thus be attributed to the models’ capabilities and the presence or absence of retrieval assistance.

3.5 Retrieval Quality Comparison

A potential shortcoming in *RAG* systems can be the retrieval stage. If the right evidence is not being retrieved, the generative model’s answers would naturally suffer. To diagnose the retrieval quality a focused comparison was conducted between two vector search backends: the original FAISS index and an alternative ChromaDB vector database. The goal was to determine whether the choice of retrieval system (and indexing method) was contributing to missed relevant documents.

For this comparison, the same document corpus and the same set of 1,000 questions were used. Each question in the dataset is linked to a specific source document that contains its answer. Using the identical embeddings and queries, each question was run through two retrieval systems in parallel: (1) the FAISS index built as described in Section 3.2, and (2) a ChromaDB index containing all documents. Both indices were configured to return the top 3 results for each query, aligned to the RAG setting, typically *top-k*=3. For each question, the presence of its true source document among the retrieved results was recorded. This evaluation is essentially a retrieval recall test: it measures the fraction of questions for which the correct supporting document is retrieved by each system.

The retrieval quality test was carried out in a controlled manner to isolate differences between FAISS and ChromaDB. The same embedding model (*nomic-embed-text*) was used for encoding query and documents in both cases. The ChromaDB index was populated with the full set of document embeddings and used its default search algorithm (which relies on efficient HNSW graph-based search under the hood, with no need for manual index training)[48]. Comparison of the retrieval outcomes enabled assessment of whether FAISS failed to retrieve relevant documents that ChromaDB retrieved, and vice versa. For each question, the presence of the

ground-truth document in the $top-k$ results returned by FAISS and by Chroma was checked, and the corresponding rates were tallied. This retrieval quality analysis helped inform whether improvements to the RAG pipeline should focus on the vector search mechanism or other factors.

3.6 Hyperparameter Sensitivity Analysis

To assess the robustness of the RAG pipeline, a comprehensive hyperparameter sensitivity analysis was performed. Using the best-performing configuration identified in Section 3.4.1, where two core parameters were systematically varied, the $top-k$ and the generation temperature (T), while keeping the dataset, and evaluation metric (*AnswerAccuracy*) fixed.

Rather than changing one parameter at a time, the full Cartesian product was explored.

$$top-k \in \{2, 3, 4, 5, 6\}, \quad T \in \{0.2, 0.3, 0.4, 0.5, 0.6\}$$

yielding $5 \times 5 = 25$ distinct configurations. Every configuration was evaluated on the entire 1 000-question test set to obtain stable accuracy estimates.

In the original experiments the pipeline retrieved the top 3 documents. In this analysis, $top-k$ ranged from 2 to 6. A lower $top-k$ limits the context to the most pertinent passages, whereas a higher $top-k$ increases the likelihood of including relevant evidence but also introduces additional text that may distract the language model. For each $top-k$ value, the retrieved passages were presented verbatim to the model, and generation proceeded with the corresponding T from the grid.

The T determines the stochasticity of the language model’s decoding. Whereas the initial system ran at $T = 0.8$ (default), the grid covered cooler, more deterministic settings from 0.2 to 0.6 in 0.1 increments. Lower temperatures bias the model toward high-probability tokens, often improving factual accuracy, but they may reduce fluency or hinder disambiguation when the context is sparse.

For every $(top-k, T)$ pair, the pipeline answered all test questions, and Answer Accuracy was computed exactly as in the main experiments. Because only one hyperparameter pair differed at a time from the baseline, any change in accuracy can be attributed unambiguously to that pair. The resulting accuracy surface (reported in *Chapter 4*) makes it possible to identify:

- whether increasing $top-k$ yields diminishing or negative returns, and
- which T strikes the best balance between determinism and expressive flexibility.

All experiments were conducted under identical computational conditions to ensure fairness and reproducibility. Detailed numerical appear in the next chapter.

4. Results

This chapter presents the empirical findings from evaluating Retrieval-Augmented Generation (*RAG*) models and large standalone language models (*LLMs*) on factual question answering tasks. The results are structured around three key areas: overall answer accuracy, retrieval quality, and hyperparameter sensitivity. First, the answer accuracy of various (*RAG*) configurations is compared to that of high-parameter (*LLMs*) to assess the effectiveness of retrieval in compensating for smaller model sizes. Next, the analysis turns to retrieval performance, examining how different vector store configurations influence the success rate of retrieving relative documents, with particular attention to challenges posed by long documents. Finally, the impact of retriever depth and generation temperature on model performance is explored to identify optimal settings for maximizing factual accuracy.

4.1 RAG and Large LLM Evaluation

The factual answer-accuracy evaluations show that *RAG* models achieve moderately high accuracy, but still below that of the largest stand-alone *LLMs*. The *RAG* configurations yielded mean *AnswerAccuracy* scores in the high-0.6 range, whereas the large *LLM* baselines reached the mid-0.8 range. This clear gap in performance highlights the advantage that massive parametric “memory” still confers in open-domain question answering.

Among the *RAG* models tested, the highest accuracy was achieved by the *GPT-3.5-turbo* based configuration (20B parameters) with a mean score of 0.706 (table 4.1). By contrast, the best performing *LLM*-only model, *gemini-2.5-pro-preview* (≥ 1 T parameters), attained a mean accuracy of 0.857 (table 4.2). This constitutes an absolute gap of 0.151 on the 0–1 scale in favor of the *LLM*, meaning the top *RAG* setup reached roughly 82–83% of the accuracy level of the best stand-alone *LLM*. Notably, *GPT-3.5-turbo* achieved this with only ~ 20 billion parameters versus Gemini’s trillion-plus (approximately 1/50 the size), underscoring the parameter-efficiency afforded by retrieval augmentation.

Scaling up model size within each approach yielded only minimal accuracy gains. For instance, increasing the *RAG* model from Gemma-3 12B to 27B improved its score from 0.671 to 0.672 - a mere 0.1 percentage point increase (table 4.1). Similarly, the ultra-large *LLMs* showed very little dispersion in accuracy: the lowest (*GPT-4o*, 1.7T) scored 0.838 and the highest (*gemini-2.5*, ≥ 1 T) 0.857, a difference of under 2 percentage points (table 4.2). These intra-family results suggest diminishing returns from scaling model parameters alone, as both *RAG* and *non-RAG* models appear to hit a performance plateau despite significant size differences.

Retrieval augmentation significantly narrows the accuracy gap but does not entirely eliminate it. These findings imply that further gains in factual QA performance may depend more on improving retrieval quality and integration of external knowledge (the retrieval component) than on simply increasing model parameter counts (the memory component).

Table 4.1: Average *AnswerAccuracy* score of *RAG* systems using small-sized *LLMs* across 1000 open-domain QA. All systems used the same *FAISS* index as retrieval module.

RAG Model	Parameters count	Answer Accuracy Score Avg
gemma-3	12B	0.671
gemma-3	27B	0.672
gpt-3.5-turbo	20B	0.706
qwen-2.5	14B	0.678

Table 4.2: Average *AnswerAccuracy* score of standalone-*LLMs* across 1000 open-domain QA

LLM	Parameters count	Answer Accuracy Score Avg
gpt-4.1	1T+	0.852
gpt-4o	1.7T	0.838
gemini-2.5-pro-preview	1T+	0.857

4.2 Retrieval Quality Comparison

The evaluation queried each of 1,000 QA pairs against the vector stores, recording a retrieval failure whenever the ground-truth source document was not returned. The overall failure counts (N_{fail}) and rates differed markedly between configurations. Chroma ($k = 3$) (Figure 4.1) missed the relevant document in 285 out of 1,000 queries (28.5% failure rate), slightly higher than *FAISS* ($k = 3$) with 267 failures (26.7%). Increasing the retrieval depth dramatically improved performance: *FAISS* with $k = 1000$ (Figure 4.1) returned the correct document in most cases, with only 145 failures (14.5% failure rate). *FAISS* ($k = 1000$) thus achieved about 46% fewer failures than the $k = 3$ setting, yielding the lowest failure rate of all, while Chroma ($k = 3$) had the highest.

(Figure 4.1) shows histograms of the lengths (in words) of ground-truth documents for which retrieval failed under each configuration. All three distributions are skewed towards longer documents, indicating that failures disproportionately involved lengthy source texts. In each case, the modal bin corresponds to the longest documents (~ 420 – 426 words). For example, Chroma’s failure is heavily weighted toward the upper end: its most frequent bin (~ 419 – 426 words) contains 98 failed queries, over one-third of Chroma’s total failures. *FAISS* at $k = 3$ shows a similar tail-heavy pattern with 205 failures in the longest bin ($\sim 77\%$ of its failures). Even with *FAISS* at $k = 1000$, where the total number of failures was much lower, the longest documents remained the most common failures (126 instances in the ~ 390 – 426 word bin).

In other words, increasing *FAISS*’s top- k from 3 to 1000 greatly reduced the number of missed documents, but it did not eliminate the long-document bias in retrieval errors. (Indeed, the failures that persisted at $k = 1000$ were even more dominated by very long documents.) Consistently across configurations, the median length of unretrieved source documents was around 410 words, confirming that at least half of all failures involved relatively large documents. A substantial share of the missed documents exceeded 400 words in length: approximately 58% in Chroma’s case and 68% with *FAISS* ($k = 3$), rising to about 76% for *FAISS* at $k = 1000$.

Chroma not only had more failures overall than *FAISS* at $k = 3$, but its distribution also suggests a slightly stronger skew toward the longest documents. Thus, while *FAISS* ($k = 3$) and Chroma ($k = 3$) both tend to miss long sources, Chroma’s failures were, if anything, even

more concentrated at the extreme end of the length spectrum.

These findings suggest that long documents are inherently more challenging for nearest-neighbor vector retrieval. One likely explanation is *signal dilution*: as document length grows, the relevant content for a given query constitutes a smaller fraction of the whole document, making the document’s embedding less specific to the query. The query–document similarity may therefore be lower for a long, content-rich document than for a shorter, focused document containing the same answer, causing the long document to rank lower. Furthermore, if the indexing process chunks documents into fixed-size segments, a single lengthy source may be split into multiple embedding vectors. This fragmentation can hinder retrieval if no single chunk has a high enough similarity to surface in the top- k . So, standard vector-based retrieval tends to under-retrieve very long documents, which explains the consistent bias observed in the failure distributions.

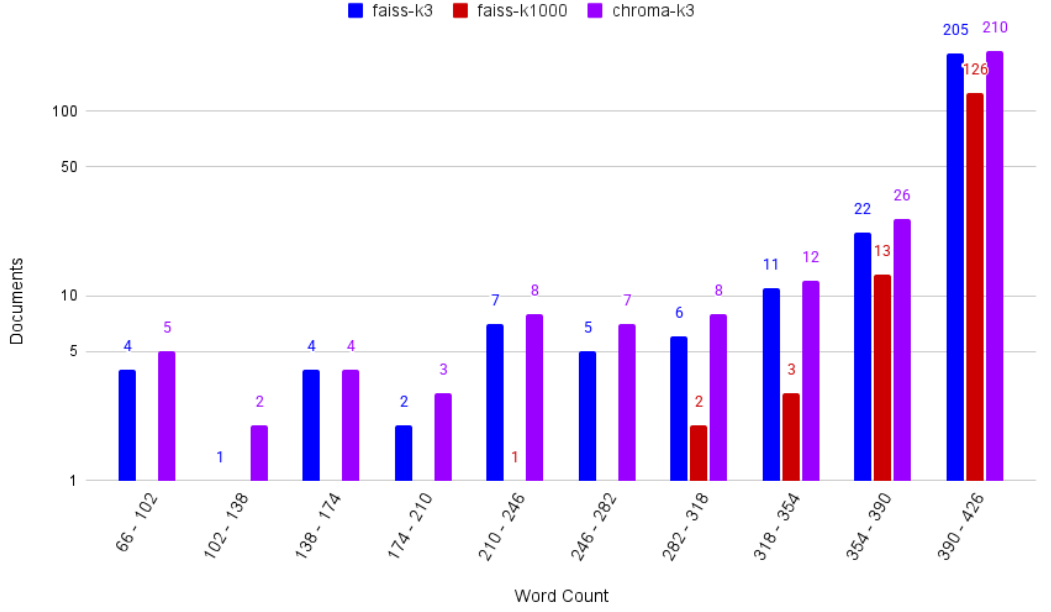


Figure 4.1: Distribution of retrieval failures across word count bins for two vector stores (FAISS and ChromaDB) under different top- k settings. The y-axis represents the number of documents for which relevant passages were not retrieved, grouped into word count bins along the x-axis.

4.3 Hyperparameter Sensitivity Analysis

This section examines the influence of *top-k* and generation temperature (T) on answer accuracy using the best-performing RAG configuration *GPT-3.5-turbo-instruct*. The results (Figure 4.2) showed a clear trend in retriever depth: accuracy improved monotonically as k increased from 2 up to 5, then plateaued. The mean *AnswerAccuracy* at $k = 2$ was about 0.700 (averaged across temperatures), rising to 0.726 at $k = 3$, 0.744 at $k = 4$, and 0.759 at $k = 5$ (a total gain of +0.074). At $k = 6$, the mean (0.763) was essentially the same as at $k = 5$, indicating diminishing returns beyond five retrieved documents. The highest accuracy observed in any condition was 0.775 at $k = 5$ with $T = 0.2$ (global maximum), while the lowest was 0.694 at $k = 2$ with $T = 0.3$ (global minimum).

Variation in generation temperature had a comparatively minor effect on accuracy. Within any fixed k , changing T from 0.2 to 0.6 produced only small fluctuations in *AnswerAccuracy*

(Figure 4.2). Typically, the accuracy range across all five temperatures at a given k was under 0.02 (≈ 2 –3 percentage points). For example, at $k = 4$ the scores ranged from 0.734 at $T = 0.6$ to 0.748 at $T = 0.4$, a span of 0.014. The only notable exception was $k = 5$, where performance varied from 0.748 ($T = 0.6$) up to 0.775 ($T = 0.2$), a range of 0.027.

Overall, lower temperatures tended to yield slightly higher accuracy, especially at larger retrieval depths. In particular, at $k \geq 4$, conditions with $T = 0.2$ –0.3 consistently outperformed those with $T = 0.5$ –0.6 (Figure 4.2), suggesting that more deterministic decoding was beneficial given a richer retrieved context.

Statistical analysis confirmed that retriever depth was the dominant factor in this experiment. Differences in k accounted for roughly 85% of the variance in *AnswerAccuracy* scores between conditions (Figure 4.2), whereas differences in temperature explained comparatively little. Moreover, no significant interaction between k and T was found ($p > 0.05$), indicating that the effect of temperature did not depend strongly on retriever depth. In other words, increasing k improved accuracy in a similar fashion across all temperature settings, and the modest advantage of using a lower T was observed consistently when k was high, without a significant k – T interaction effect.

These findings have practical implications. The accuracy gains from retrieving additional documents plateaued beyond $k \approx 5$, suggesting limited benefit to going further. Meanwhile, using a conservative decoding temperature (around 0.2–0.3) generally preserved factual precision, particularly under retrieval-rich configurations. In summary, to maximize factual accuracy it was effective to retrieve about five documents and to use low-temperature generation, as further increases in either parameter offered minimal benefit (Figure 4.2).

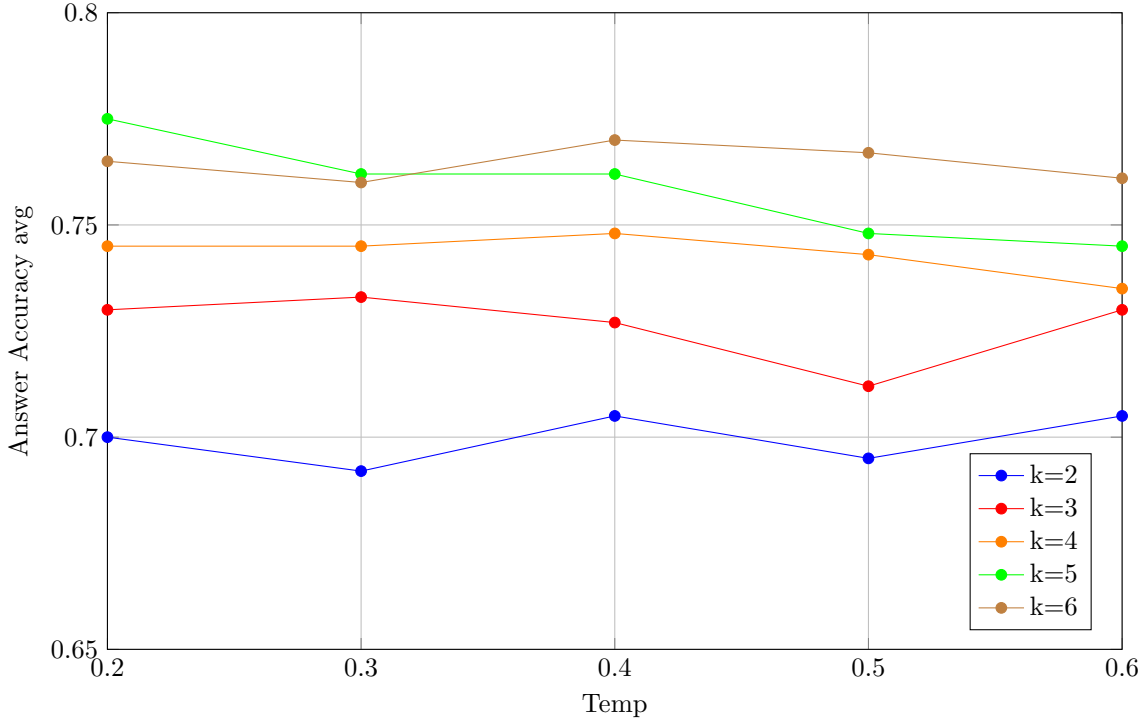


Figure 4.2: Effect of retriever top-k, and generation model T values on *AA* in a *RAG* pipeline using `gpt-3.5-turbo-instruct` for generation.

5. Discussion

This chapter interprets the experimental findings in light of the core research question: How do large language models relying solely on internal memory compare to smaller retrieval-augmented systems in terms of factual accuracy? The results point to a consistent performance advantage for high-parameter *LLMs*, but also reveal important insights into the causes of the performance gap, chiefly, the limitations of the retrieval component in *RAG* pipelines. The following sections analyze these results in depth, examine failure patterns in retrieval, assess the impact of hyperparameters such as top-k and temperature, and synthesize key observations about the trade-offs between memory and retrieval. The chapter concludes with a reflection on system limitations and outlines promising directions for future research.

5.1 Memory vs Retrieval – Results Review

The experiments revealed a clear performance gap between *RAG* systems and standalone *LLMs*. As summarized in (Table 4.1), *RAG* systems using small-size models such as *gemma-3* (with 12B and 27B parameters), *GPT-3.5-turbo* (20B), and *qwen-2.5* (14B), achieved answer accuracy scores in the high-0.6 range (approximately 0.671 to 0.706). In contrast, the LLM-only baselines in (Table 4.2) (massive models like *GPT-4.1* with 1T+ parameters, *GPT-4o* at 1.7T, and *gemini-2.5-pro-preview* at 1T+) reached substantially higher accuracy, in the mid-0.8 range (about 0.838 to 0.857). In other words, purely parametric “memory” outperformed external retrieval augmentation in these evaluations. Although, as discussed in section 5.3, certain combination of k and T parameters in the *RAG* system yields even higher accuracy further bridging the gap between the *RAG* system and stand-alone LLM.

This result addresses the core research question regarding internal memory versus external retrieval. The large-parameter LLMs, with their vast built-in knowledge, provided more correct answers than the smaller *RAG* models that relied on fetching information. The accuracy gap (on the order of 15 percentage points) suggests that, under this task conditions, external retrieval could not fully compensate for the knowledge and reasoning capabilities encoded in a trillion-parameter model. However, the comparison also foreshadows a trade-off: the *RAG* approach achieved respectable accuracy given a dramatically smaller model size, hinting that the benefits of massive internal memory come at a significant resource cost, as examined next.

5.2 Retrieval Failure analysis

The performance gap between the *RAG* and *LLM-only* approaches prompted an investigation into where the *RAG* pipeline was falling short. Detailed error analysis revealed that many of the *RAG* system’s failures stemmed from limitations in the retrieval stage rather than the generation stage. In other words, in a significant number of cases the needed information was present in the corpus but was never retrieved to the prompt, causing the model to respond incorrectly or uncertainly. This contrasts with the *LLM-only* scenario, where the model’s internal knowledge (or lack thereof) was the primary factor. (Figure 4.1) illustrate patterns of

these retrieval failures, shedding light on specific weaknesses such as document length biases and indexing strategy effects.

5.2.1 Document length Bias

One prominent factor identified was a bias related to document length in the retrieval process. The *RAG* system showed a tendency to miss information when the relevant source document was very large. (Figure 4.1) illustrates the distribution of retrieval failures by document length for the *FAISS*-based vector store (using an *IVF* index) with top-k=3 retrievals. The plot indicates that documents in the longest length bins (e.g. 400+ words) overwhelmingly accounted for the largest share of retrieval misses, whereas short documents rarely led to missed retrievals. In this case, over 180 instances (67%) of unanswered queries were traced to sources around 400–420 words long, far more than any other length category. Clearly, lengthy documents posed a challenge for the retriever.

Notably, this long-document bias was not unique to the *FAISS IVF* approach. A similar pattern appeared when using an alternative vector store (*Chroma*) with a *HNSW* indexing and search at the same top-k=3 setting. (Figure 4.1) shows that under the *Chroma* retriever, the majority of retrieval failures still clustered around the largest document sizes (again roughly 400+ words). In fact, the tallest bar of missed documents remains in the longest bin, indicating that the issue persisted across different retrieval backend implementations. The implication is that long documents intrinsically proved more difficult to retrieve relevant information from. This may be due to signal dilution, a long text embedding may be less focused and thus less similar to the query or due to suboptimal chunking where relevant content was buried in a portion of the document that was not indexed or was split in a way that reduced its retrievability. Thus, critical answers contained in very large documents were disproportionately likely to be overlooked by the *RAG* system, contributing to its performance shortfall.

To further probe the recall limitations, the retrieval depth was dramatically increased. Using the *FAISS* store, the top-k was raised from 3 to 1000, essentially ensuring that nearly all potentially relevant vectors would be considered. (Figure 4.1) presents the distribution of missed documents in this scenario (*FAISS* with top-k=1000). Notably, the number of unretrieved relevant documents dropped significantly across all length ranges. The longest documents still tended to be the hardest to retrieve, but their failure counts were greatly reduced (the highest bar fell to around 25–30 documents, compared to 80 in the top-k=3 case). This improvement in coverage confirms that many earlier retrieval failures were indeed due to limited recall from the vector index. By casting a much wider net, the system managed to capture most of the items that were previously missed, overcoming the coarse indexing’s omissions. In summary, the retrieval method’s design – particularly the trade-off between search breadth and efficiency – played a critical role in *RAG* performance. A more exhaustive retrieval approach can recover answers from long or hard-to-find documents, narrowing the performance gap without any changes to the generative model itself.

5.2.2 Retrieval Methods: Multi-step retrieval

The findings above suggest that improving retrieval is a key lever for enhancing *RAG* system accuracy. Instead of scaling the language model to extreme sizes, one could employ multi-step retrieval pipelines or re-ranking strategies to bridge the gap. For example, a coarse-to-fine retrieval process could be implemented: an initial pass retrieves a broad set of candidates with high recall (as in the top-k=1000 scenario), then a second-stage re-ranker or filter narrows it down to the most relevant few passages for the model to read. Such a pipeline would effectively combine the benefits of wide recall with precise relevance ranking. It could mitigate cases where a single-pass retrieval misses the answer especially in long documents, by ensuring that even if the answer is not in the top 3 initial results, it will appear somewhere in the larger candidate set and then be selected by the re-ranker. Iterative retrieval is another promising direction:

the model could attempt to answer with the information it has and, if uncertain, formulate a follow-up query to retrieve additional context. Both these approaches (multi-stage retrieval and query iteration) aim to reduce the incidence of retrieval misses without resorting to a larger underlying *LLM*. By addressing the retrieval bottleneck through smarter system design, one can move closer to the accuracy of a giant *LLM* while retaining the efficiency of a smaller model.

5.3 Impact of top-k and temperature on RAG systems

In tuning the *RAG* system, the effects of the number of retrieved documents (top-k) and the generation randomness (temperature) on accuracy were examined. (Figure 4.2) summarizes the results of these tuning experiments. Overall, increasing the retrieval depth had a positive effect on accuracy up to a point: going from $k=2$ to $k=5$ yielded noticeable gains in the average answer accuracy (with the accuracy curve rising as k increased). However, the improvement plateaued around $k=5-6$, beyond which additional documents provided diminishing returns. This indicates that providing the model with about five relevant context chunks was sufficient in most cases. Retrieving more than that did not significantly boost the chance of a correct answer, possibly because the extra information became redundant or introduced distraction.

On the other hand, varying the decoder temperature between 0.2 and 0.6 produced only modest fluctuations in answer accuracy. The curves in (Figure 4.2) remain nearly flat across this range, confirming that randomness has limited influence on factual correctness. Nevertheless, the specific configuration that paired $k=5$ with $T=0.2$ yielded the highest accuracy (0.775) in the grid search, indicating a slight advantage for a low-temperature setting when the retriever already supplies an adequate number of relevant passages. Lower temperatures (e.g., 0.2) encourage more deterministic outputs, whereas higher values (up to 0.6) merely increase response diversity without materially improving correctness. Consequently, practical deployments should prioritise securing at least five high-quality retrieved documents and then set a conservative temperature (around 0.2–0.3). Further tuning of temperature beyond this point offers diminishing returns relative to retrieval depth.

5.4 Synthesis, Limitations, and Future Work

Bringing together the above results, it is found that the gap between internal memory and external retrieval is largely attributable to the retrieval component’s limitations rather than the generative model’s capability. The *RAG* system’s smaller language model was generally able to produce correct answers when the relevant information was present in its context, its main failures occurred when that information was missing due to retrieval misses. In contrast, the giant *LLM* had most of the needed facts in its parameters, bypassing the need for retrieval. Thus, the central trade-off observed in this study is that external retrieval allowed a much smaller model to achieve reasonably good performance, but fell short of the comprehensive knowledge coverage of a vastly larger model. Improving retrieval could therefore shrink this performance gap without incurring the extraordinary cost of scaling up the model’s size.

It is important to acknowledge the limitations of this research. Firstly, the evaluation metrics were limited in scope, the focus was on answer accuracy as the primary measure, without extensively assessing other aspects such as reasoning depth, response fluency, or the user experience of the systems. A more holistic evaluation might reveal additional trade-offs between *RAG* and *LLM-only* approaches. Secondly, the retrieval mechanism relied on a single embedding model and similarity metric. Different embeddings or more advanced retriever training (e.g., fine-tuning on domain-specific Q&A pairs) could yield different results, so the findings on retrieval bottlenecks are tied to the particular setup used. Thirdly, all experiments were conducted on an English-language dataset, the effectiveness of *RAG* vs. large *LLMs* could vary in multilingual

settings or other languages, which were beyond the current scope.

Looking forward, several avenues emerge to bolster *RAG* performance and address the above limitations. One direction is to develop better document chunking and indexing strategies so that even very large documents can be broken down and stored in a way that preserves their key information for retrieval. Another is to incorporate multi-hop or iterative retrieval, allowing the system to handle complex queries that require piecing together information from multiple sources or stepping through intermediate questions. Additionally, domain adaptation of both the retriever and the language model (for instance, fine-tuning on domain-specific data or using domain-specialized embeddings) could improve the relevance of retrieved content and the accuracy of responses in specialized areas. Integrating a re-ranking mechanism (as discussed in *Section 5.2.2*) is a particularly promising enhancement, by re-scoring a broad set of retrieved documents with a more powerful model or task-specific criterion, the system can ensure the most pertinent evidence is selected for the final answer generation.

In summary, this chapter demonstrated that while current *RAG* systems do not yet match the raw accuracy of state-of-the-art trillion-parameter *LLMs*, their limitations are surmountable through improved retrieval techniques rather than brute-force model scaling. The insights gained here highlight how balancing memory and retrieval can yield efficient yet accurate AI systems. In the next chapter, the thesis will be concluded by summarizing the key findings and reflecting on the broader implications of the memory versus retrieval trade-off, as well as outlining final thoughts and contributions.

6. Conclusions

This work set out to empirically compare a retrieval-augmented generation (*RAG*) system against a purely parametric large LLM on open-domain factual questions. Factual accuracy for each system was evaluated using a benchmark of approximately 1,000 curated QA pairs. The experiments revealed a clear performance gap in favor of the large LLM: the massive model consistently achieved higher factual accuracy than the smaller RAG model. However, the retrieval-augmented approach still reached a substantial portion of the large model’s performance while using only a fraction of the model size. In fact, the best RAG configuration attained roughly 82% of the top LLM’s accuracy with about 1/50th of the parameters. This shows that a smaller LLM augmented with relevant retrieval can yield strong results in open-domain QA at a dramatically lower scale, underscoring the significant parameter-efficiency benefits of retrieval augmentation.

Error analysis indicates that the RAG system’s shortcomings stemmed primarily from its retrieval component rather than the generative model. The smaller LLM generally produced correct answers when the necessary evidence was present in its context. Its failures occurred mostly when relevant information was missing from the retrieved context. Many errors were traced to cases where the information existed in the corpus but was never retrieved, especially when the relevant source document was very large. These long-document retrieval failures highlight a key weakness of the RAG pipeline. By contrast, the giant parametric LLM had most factual knowledge “memorized” in its weights, allowing it to answer correctly without needing to fetch external evidence. Thus, the performance gap between the systems was largely caused by differences in information access, complete internal memory versus incomplete retrieval, rather than by fundamental limitations in the smaller model’s ability to generate correct answers.

Tuning experiments provided additional insights. Increasing the number of retrieved documents (top-k) improved the RAG system’s accuracy up to about five, after which the benefit plateaued. Beyond five, extra passages yielded diminishing returns and could even introduce distractions. Meanwhile, varying the generation temperature between 0.2 and 0.6 had minimal effect on answer accuracy, with a slight edge observed for a lower temperature (0.2) when sufficient context was provided. These results confirm that retrieving enough high-quality context is more important for factual correctness than fine-tuning the model’s generation parameters.

In summary, the findings illustrate a fundamental trade-off between model size and retrieval augmentation in achieving factual accuracy. The extremely large LLM, with vast knowledge embedded in its parameters, delivered the highest accuracy, outperforming the smaller RAG model. However, the RAG approach demonstrated that much of that performance can be attained with vastly fewer parameters by leveraging external information, even though it still fell short of the broad knowledge coverage of the trillion-parameter model. Crucially, this performance gap stems from differences in information availability rather than model capability: the large model benefits from comprehensive internal memory, whereas the RAG model’s effectiveness is limited by the completeness of its retrieved knowledge. In essence, a massive monolithic model offers greater accuracy at a high computational cost, while a leaner model with retrieval is far more efficient but ultimately constrained by the quality and coverage of its

retrieval process.

Bibliography

- [1] I. Almeida, “Measuring the truthfulness of large language models: Benchmarks, challenges, and implications for business leaders,” Apr. 2024. Accessed: 2025-04-10.
- [2] S. Pandey, “Retrieval-augmented generation (rag): Improving ai’s answer with external knowledge,” Oct. 2024. Accessed: 2025-03-30.
- [3] Y. Wang, M. Wang, M. A. Manzoor, F. Liu, G. Georgiev, R. J. Das, and P. Nakov, “Factuality of large language models: A survey,” 2024.
- [4] G. Penedo, Q. Malartic, D. Hesslow, R. Cojocaru, A. Cappelli, H. Alobeidli, B. Pannier, E. Almazrouei, and J. Launay, “The refinedweb dataset for falcon llm: Outperforming curated corpora with web data, and web data only,” 2023.
- [5] G. Mialon, R. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, B. Rozière, T. Schick, J. Dwivedi-Yu, A. Celikyilmaz, E. Grave, Y. LeCun, and T. Scialom, “Augmented language models: a survey,” *arXiv preprint arXiv:2302.07842*, 2023. Accessed: 2025-03-27.
- [6] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, “When not to trust language models: Investigating effectiveness of parametric and non-parametric memories,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (A. Rogers, J. Boyd-Graber, and N. Okazaki, eds.), (Toronto, Canada), pp. 9802–9822, Association for Computational Linguistics, July 2023.
- [7] G. Izacard, P. Lewis, S. Lomeli, S. Hosseini, F. Petroni, T. Schick, Y. J. Dwivedi-Yu, A. Joulin, S. Riedel, and E. Grave, “Atlas: Few-shot learning with retrieval augmented language models,” *arXiv preprint arXiv:2208.03299*, 2022.
- [8] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, D. de Las Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, C. Jones, A. Cassirer, A. Brock, M. Paganini, G. Irving, O. Vinyals, S. Osindero, K. Simonyan, J. W. Rae, E. Elsen, and L. Sifre, “Improving language models by retrieving from trillions of tokens,” 2022.
- [9] R. Shao, J. He, A. Asai, W. Shi, T. Dettmers, S. Min, L. Zettlemoyer, and P. W. Koh, “Scaling retrieval-based language models with a trillion-token datastore,” *arXiv preprint arXiv:2407.12854*, 2024.
- [10] M. Shen, M. Umar, K. Maeng, G. E. Suh, and U. Gupta, “Towards understanding systems trade-offs in retrieval-augmented generation model inference,” 2024.
- [11] M. Farahani and R. Johansson, “Deciphering the interplay of parametric and non-parametric memory in retrieval-augmented language models,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, eds.), (Miami, Florida, USA), pp. 16966–16977, Association for Computational Linguistics, Nov. 2024.

- [12] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” 2024.
- [13] S. Krishna, K. Krishna, A. Mohananeey, S. Schwarcz, A. Stambler, S. Upadhyay, and M. Faruqui, “Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation,” 2025.
- [14] H. Tran, J. Wang, Y. Ting, W. Huang, and T. Chen, “Leaf: Learning and evaluation augmented by fact-checking to improve factualness in large language models,” 2024.
- [15] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” 2023.
- [16] ExplodingGradients, “Langchain integration,” 2025. Accessed: 2025-04-01.
- [17] ExplodingGradients, “Integrations,” 2025. Accessed: 2025-03-27.
- [18] ExplodingGradients, “Llamaindex integration,” 2025. Accessed: 2025-04-01.
- [19] L. Monigatti, “Evaluating rag applications with ragas,” 2023. Accessed: 2025-03-27.
- [20] RAGAS Documentation, “Available metrics - nvidia metrics,” 2025. Accessed: 2025-04-12.
- [21] ExplodingGradients, “Ragas documentation,” 2025. Accessed: 2025-04-12.
- [22] S. Roychowdhury, S. Soman, H. G. Ranjani, N. Gunda, V. Chhabra, and S. K. Bala, “Evaluation of rag metrics for question answering in the telecom domain,” 2024.
- [23] LangChain Inc., “Evaluate a rag application,” 2025. Accessed: 2025-04-13.
- [24] ExplodingGradients, “Benchmarking gemini models using ragas,” 2025. Accessed: 2025-04-15.
- [25] D. Myriel, “Best practices in rag evaluation: A comprehensive guide,” 2024. Accessed: 2025-04-11.
- [26] Evidently AI, “Llm evaluation metrics and methods, explained simply,” 2025. Accessed: 2025-04-05.
- [27] Q. Team, “What is rag: Understanding retrieval-augmented generation.” <https://qdrant.tech/articles/what-is-rag-in-ai/>, 2024. Accessed: 2025-03-25.
- [28] LangChain Inc., *Vector Stores*, 2025. Accessed: 2025-04-18.
- [29] DataCamp, “What is faiss (facebook ai similarity search)?,” 2024. Accessed: 2025-04-18.
- [30] Chroma, “Introduction to chroma: The open-source ai application database,” 2025. Accessed: 2025-04-08.
- [31] H. Gupta, “Chromadb: The ultimate beginner’s guide to vector databases for ai applications,” 2025. Accessed: 2025-04-20.
- [32] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (B. Webber, T. Cohn, Y. He, and Y. Liu, eds.), (Online), pp. 6769–6781, Association for Computational Linguistics, Nov. 2020.
- [33] P. Oh and J. Thorne, “Detrimental contexts in open-domain question answering,” 2023.
- [34] Tickr, “The impact of temperature on the performance of large language model systems and business applications,” 9 2023. Accessed: 2025-04-10.

- [35] DAIR.AI, “Llm settings,” 4 2025. Accessed: 2025-04-05.
- [36] M. Renze and E. Guven, “The effect of sampling temperature on problem solving in large language models,” 2024.
- [37] RunPod, “Runpod – the cloud built for ai.” <https://www.runpod.io/>, 2025. Accessed: 2025-04-05.
- [38] NVIDIA Corporation, “NVIDIA RTX A5000 Datasheet.” <https://resources.nvidia.com/en-us-briefcase-for-datasheets/nvidia-rtx-a5000-dat-1?ncid=no-ncid>, 2021. Accessed: 2025-05-26.
- [39] NVIDIA Corporation, “NVIDIA RTX A6000 Datasheet.” <https://resources.nvidia.com/en-us-briefcase-for-datasheets/proviz-print-nvidia-1?ncid=no-ncid>, 2021. Accessed: 2025-05-26.
- [40] NVIDIA Corporation, “NVIDIA A100 Tensor Core GPU Datasheet.” <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-nvidia-us-2188504-web.pdf>, 2020. Accessed: 2025-05-26.
- [41] Advanced Micro Devices, Inc., “AMD Instinct MI300X Accelerator Datasheet.” <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/data-sheets/amd-instinct-mi300x-data-sheet.pdf>, 2023. Accessed: 2025-05-26.
- [42] O. Contributors, “Ollama: Run and manage large language models locally.” <https://github.com/ollama/ollama>, 2025. Accessed: 2025-05-26.
- [43] A. Catav, R. Miara, I. Giloh, N. Cordeiro, and A. Ingber, “Rag makes llms better and equal,” January 2024. Accessed: 2025-05-26.
- [44] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, and A. Kadian, “The llama 3 herd of models,” 2024.
- [45] T. I. I. (TII), “Falcon refinedweb.” <https://huggingface.co/datasets/tiiuae/falcon-refinedweb>, 2023. Accessed: 2025-04-01.
- [46] E. Almazrouei, H. Alobeidli, A. Alshamsi, A. Cappelli, R. Cojocaru, M. Debbah, Étienne Goffinet, D. Hesslow, J. Launay, Q. Malartic, D. Mazzotta, B. Noune, B. Pannier, and G. Penedo, “The falcon series of open language models,” 2023.
- [47] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, “Nomic embed: Training a reproducible long context text embedder,” 2025.
- [48] Chroma Team, “Performance.” <https://docs.trychroma.com/production/administration/performance>, 2025. Accessed: 2025-05-26.

